Check Number Is prime or not in O(sqrt(n)):

```
bool isPrime(int n) {
    if (n <= 1)
        return false;
    if (n <= 3)
        return true;
    if (n % 2 == 0 || n % 3 == 0)
        return false;

    for (int i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0)
            return false;
    }
    return true;
}
```

Sieve O(nlog(n)log(n)):

```
const int N = 1e6 + 5;
vector<bool> isPrime(N, true);

void sieve(int n) {
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i * i <= n; ++i) {
        if (isPrime[i]) {
            for (int j = i * i; j <= n; j += i)
                isPrime[j] = false;
        }
    }
}
```

Euler Sieve O(n):

```
const int N = 1e6 + 5;
vector<int> primes;
vector<bool> isPrime(N, true);

void eulerSieve(int n) {
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i <= n; ++i) {
```

```
            if (isPrime[i]) primes.push_back(i);
            for (int p : primes) {
                if (i * p > n) break;
                isPrime[i * p] = false;
                if (i % p == 0) break;
            }
        }
    }
```

Count Divisors of One Number O(√n):

```
int countDivisors(int n) {
    int cnt = 0;
    for (int i = 1; i * i <= n; ++i) {
        if (n % i == 0) {
            cnt++; // i
            if (i != n / i) cnt++; // n / i (avoid double count for squares)
        }
    }
    return cnt;
}
```

Count Divisors of All Numbers from 1 to n O(n log n):

```
const int N = 1e6 + 5;
vector<int> divisors(N, 0);

void countAllDivisors(int n) {
    for (int i = 1; i <= n; ++i)
        for (int j = i; j <= n; j += i)
            divisors[j]++;
}
```

Prime Factorization in O(√n):

```
map<int, int> primeFactorize(int n) {
    map<int, int> factors;
    for (int i = 2; i * i <= n; ++i) {
        while (n % i == 0) {
            factors[i]++;
```

```
            n /= i;
        }
    }
    if (n > 1) factors[n]++;
    return factors;
}
```

Prime Factorization in O(n log n):

```
const int N = 1e6 + 5;
vector<int> spf(N); // smallest prime factor

void sieve() {
    for (int i = 0; i < N; i++)spf[i] = i;

    for (int i = 2; i * i < N; i++) {
        if (spf[i] == i) {
            for (int j = i * i; j < N; j += i)
                if (spf[j] == j)spf[j] = i;
        }
    }
}
vector<int> getFactorization(int x) {
    vector<int> factors;
    while (x != 1) {
        factors.push_back(spf[x]);
        x /= spf[x];
    }
    return factors;
}
```

Prime Factorization using all priems pre calc factorize 1e9 more fast:

```
vector<int> factorize(int x, const vector<int>& primes) {
    vector<int> res;
    for (int p : primes) {
        if (1LL * p * p > x) break;        // Stop if prime^2 > x
        if (x % p == 0) {
            res.push_back(p);              // p is a prime factor
            while (x % p == 0) x /= p;     // Remove all occurrences of p
        }
    }
```

```
    if (x > 1) res.push_back(x);          // x is a large prime factor
    return res;
}
```

Fast Power (O(log b)):

```
long long fastPower(long long a, long long b) {
    long long res = 1;
    while (b > 0) {
        if (b & 1)
            res *= a;
        a *= a;
        b >>= 1;
    }
    return res;
}
```

Fast Power with Mod (O(log b)):

```
long long fastPowerMod(long long a, long long b, long long mod) {
    long long res = 1;
    a %= mod;
    while (b > 0) {
        if (b & 1)
            res = (res * a) % mod;
        a = (a * a) % mod;
        b >>= 1;
    }
    return res;
}
```

GCD , LCM:

```
int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}

int lcm(int a, int b) {
```

```
    return a / gcd(a, b) * b;
}
```

---

nPr , nCr:

```
const int M = 3e6+10;
int mod = 1e9+7;
int add ( int a , int b)
{
    return (a + b) % mod;
}
int mul ( int a , int b)
{
    return 1LL * a * b % mod;
}
int fp( int b , int p)
{
    if(!p)
        return 1;
    int temp = fp(b,p/2);
    temp = mul(temp,temp);
    if(p&1)
        temp = mul(temp,b);
    return temp;
}

int fact[M], inv[M];
void build()
{
    fact[0] = fact[1] = inv[0] = inv[1] = 1;
    for (int i = 2; i < M; i++)
    {
        fact[i] = fact[i - 1] * i % mod;
        inv[i] = inv[mod % i] * (mod - mod / i) % mod;
    }
    for (int i = 2; i < M; i++)
        inv[i] = inv[i] * inv[i - 1] % mod;
}
int nCr( int n , int r)
{
    return mul(fact[n],mul(inv[n-r],inv[r]));
}
int nPr(int n, int r)
{
    int ans = nCr(n, r); ans *= fact[r];
    ans %= mod;
    return ans;
}
```

Compute ϕ(n):

```
int phi(int n) {
    int res = n;
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            res -= res / i;
        }
    }
    if (n > 1)
        res -= res / n;
    return res;
}
```

Compute ϕ(n) for range:

```
const int N = 1e6 + 5;
int phi[N];

void computeTotients() {
    for (int i = 0; i < N; i++) phi[i] = i;
    for (int i = 2; i < N; i++) {
        if (phi[i] == i) {
            for (int j = i; j < N; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}
```
